

Feature Storages at Scale

RecSys Course

2026

“Any sufficiently complicated ML system contains an ad hoc, informally specified, bug-ridden, slow implementation of a feature store.”

— variation of Greenspun’s Tenth Rule

Roadmap: The Evolution of Feature Storage

1. Static & updatable in-memory data storages
2. Contextual features & logging them
3. Sampling logs at scale
4. Reducing log sizes & feature computation separation
5. Logging explosion for mature ML teams
6. Online/offline skew & how to fix
7. Unified feature computation library
8. Centralized feature store

Theme: each stage solves a bottleneck from the previous one — and introduces a new one.

Static Feature File

Setup: precomputed features stored in an embedded file, shipped with the service binary.

- Scalar/categorical features → key-value trie file (keyvi, LMDB)
- Vector embeddings → ANN index file (Faiss, LanceDB)
- File is rebuilt offline, deployed as part of the service release

Pros

- Zero network latency (process-local reads)
- Simplest possible architecture
- No external dependencies at serving time

Cons

- Features go stale between deploys
- No contextual or real-time features
- File size limited by node RAM/disk

Embedded KV Stores & Vector Libraries

Key-value stores:

- **keyvi** — FST-based, 5–10× compression, immutable; static dictionaries
- **LMDB / MDBX** — CoW B+ tree, zero-copy reads; read-heavy <100 GB
- **RocksDB** — LSM-tree, streaming updates, compression; general-purpose, TB-scale
- **LevelDB** — LSM-tree, tiny footprint; prototyping only

Vector libraries:

- **Faiss** (Meta) — IVF, PQ, HNSW, GPU support. Library, not a database.
- **LanceDB** — embedded columnar vector DB, Lance format, zero-copy.

Embedded KV Stores & Vector Libraries

Key-value stores:

- **keyvi** — FST-based, 5–10× compression, immutable; static dictionaries
- **LMDB / MDBX** — CoW B+ tree, zero-copy reads; read-heavy <100 GB
- **RocksDB** — LSM-tree, streaming updates, compression; general-purpose, TB-scale
- **LevelDB** — LSM-tree, tiny footprint; prototyping only

Vector libraries:

- **Faiss** (Meta) — IVF, PQ, HNSW, GPU support. Library, not a database.
- **LanceDB** — embedded columnar vector DB, Lance format, zero-copy.

Bottleneck: features are frozen until next deploy → need periodic refresh.

Daily File Refresh

Bottleneck: features go stale between service releases.

Solution: decouple feature updates from code deploys.

- Daily batch job (Spark / Airflow) recomputes feature files
- New file pushed to service nodes (sidecar pattern or hot-reload)
- Historical files archived to cold storage (S3 / GCS)
- Manual code assembles training pool from historical files

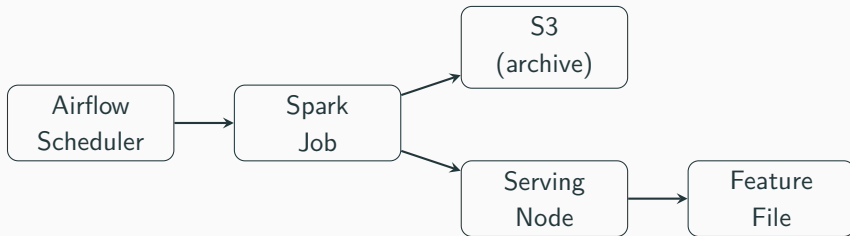
Pros

- Daily freshness
- Historical data preserved
- Still simple serving path

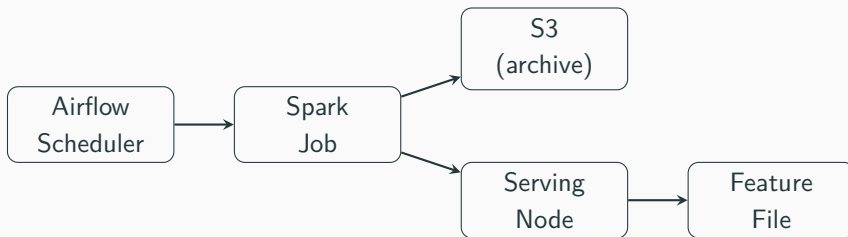
Cons

- Manual pool assembly code
- No contextual features
- No schema, no versioning

Daily File Refresh: Architecture



Daily File Refresh: Architecture



Bottleneck: offline-only features cannot capture request context → need logging.

Contextual Features Emerge

Bottleneck: precomputed features miss request-time context.

Examples of contextual features:

- Time of day, day of week
- Device type, screen size, app version
- Position in the recommendation list
- Previous actions in the current session
- Cross-feature interactions computed at serving time

Consequence: we can no longer rely solely on offline-computed files.

We must **log serving outputs** (features + context) to use them for training.

Logging Format: Schema vs Raw JSON

- **Raw JSON** — flexible, universal parsers; verbose, brittle evolution, no types
- **Schema-first (Protobuf / Avro / Thrift)** — compact binary, backward/forward compatible, strong types, code generation

Industry: schema-first is the standard for production logging.

- Netflix: Thrift for snapshot data serialization
- Uber: Avro for feature serialization in Michelangelo
- General trend: Protobuf (gRPC ecosystem, smaller footprint)

Where to Write Logs

Option A: Rotating files → S3

- Write to local disk, rotate hourly
- Upload rotated files to S3/GCS
- Simple, cheap, no infra dependency
- Batch-only consumption
- Risk: data loss if node dies before upload

Option B: Kafka topic

- Write to Kafka from the service
- Multiple consumers: S3 sink, Flink, etc.
- Real-time consumption possible
- Replay from any offset
- Higher infra cost, Kafka ops overhead

Recommendation: start with rotating files → S3. Move to Kafka when you need real-time consumers or multiple downstream pipelines.

Can't Log Everything

Bottleneck: log volume grows linearly with traffic.

The math:

- $100\text{M requests/day} \times 500 \text{ candidates} \times 200 \text{ features} \times 4 \text{ bytes}$
- $\approx 40 \text{ TB/day}$ of raw feature logs
- Storage cost at \$23/TB/month: \$11K/year just for 1 month retention

Solution: sample serving outputs instead of logging everything.

- 1–10% sample often sufficient for training
- Must be **representative** — biased samples $\rightarrow$ biased models

Stratified Sampling Strategies

Goal: preserve distribution across important dimensions.

Dimensions to stratify by:

- Experiment groups (A/B test arms)
- Geography / region / country
- Device type (mobile, TV, web, tablet)
- User activity level (cold-start, active, power users)

Techniques:

- **Consistent hashing** on user_id → deterministic, reproducible
- **Reservoir sampling** → uniform over a stream
- **Netflix approach:** Spark SQL selects context samples from Hive, merges across experiments, persists to S3 with experiment identifiers

Hundreds of Features, Log Only Used

Bottleneck: to save resources, we compute and log only the features that the *current* model uses.

The problem:

1. Researcher has idea for a new feature
2. Feature is not currently computed or logged
3. Must enable computation + logging in production
4. Wait days or weeks for data to accumulate
5. Only then can run offline experiment

Impact: experiment cycle stretches from days to weeks/months.

Root cause: tight coupling between serving features and training data.

Hundreds of Features, Log Only Used

Bottleneck: to save resources, we compute and log only the features that the *current* model uses.

The problem:

1. Researcher has idea for a new feature
2. Feature is not currently computed or logged
3. Must enable computation + logging in production
4. Wait days or weeks for data to accumulate
5. Only then can run offline experiment

Impact: experiment cycle stretches from days to weeks/months.

Root cause: tight coupling between serving features and training data.

Bottleneck: logging features is too slow for experimentation → log raw inputs instead.

Log Raw Inputs, Not Features

Key insight: decouple feature *definition* from feature *logging*.

Instead of logging computed features:

- Log **raw data snapshots** from upstream services at request time
- User stats, embeddings, action histories, model outputs
- Store snapshots with timestamps in cold storage
- Compute **any** features offline from these snapshots

Benefits:

- New feature idea → compute immediately from historical snapshots
- No production changes needed for experimentation
- Same raw data supports many different feature definitions

Netflix DeLorean: Time Travel for Features

Architecture:

1. **Context selection:** Spark SQL samples contexts from Hive (stratified)
2. **Snapshot:** Prana sidecar fetches data from microservices (Viewing History, My List, Predicted Ratings, ...), stores in S3/Parquet
3. **Time Travel API:** given a timestamp, retrieves the snapshot, reconstructs POJOs that microservices would have returned
4. **Feature generation:** Spark runs *the same feature encoder code* on snapshot data as would run online

Key property: *one feature encoder implementation* used both offline (on snapshots) and online (on live service calls). Eliminates online/offline skew by design.

Netflix DeLorean: Time Travel for Features

Architecture:

1. **Context selection:** Spark SQL samples contexts from Hive (stratified)
2. **Snapshot:** Prana sidecar fetches data from microservices (Viewing History, My List, Predicted Ratings, ...), stores in S3/Parquet
3. **Time Travel API:** given a timestamp, retrieves the snapshot, reconstructs POJOs that microservices would have returned
4. **Feature generation:** Spark runs *the same feature encoder code* on snapshot data as would run online

Key property: *one feature encoder implementation* used both offline (on snapshots) and online (on live service calls). Eliminates online/offline skew by design.

Bottleneck: every team builds their own logging → explosion of spaghetti.

Logging Explosion

Bottleneck: success breeds chaos.

What happens:

- Every team/service starts logging “their” raw data
- Different formats, schemas, storage locations
- Duplicated data across services (same user stats logged 5 times)
- No coordination on retention, sampling, or schema evolution
- Spaghetti code: each team writes custom ETL to assemble training pools

Consequences:

- Storage costs grow super-linearly
- Debugging data issues requires cross-team archeology
- Onboarding new ML engineers takes weeks

Logging Explosion

Bottleneck: success breeds chaos.

What happens:

- Every team/service starts logging “their” raw data
- Different formats, schemas, storage locations
- Duplicated data across services (same user stats logged 5 times)
- No coordination on retention, sampling, or schema evolution
- Spaghetti code: each team writes custom ETL to assemble training pools

Consequences:

- Storage costs grow super-linearly
- Debugging data issues requires cross-team archeology
- Onboarding new ML engineers takes weeks

The most dangerous failure mode in ML systems.

Training-serving skew (a.k.a. feature drift):

- Feature code updated in the online service
- Offline pipeline still uses the old version
- Model trained on stale feature distribution
- Predictions degrade silently — no error, just worse metrics

Common drift examples:

- Normalization added online, missing offline
- Feature window changed (7d $\rightarrow$ 14d) online, offline still 7d
- New categorical values appear online, unknown to offline encoder
- Timestamp timezone bug: UTC online, local time offline

Detecting and Preventing Skew

Detection:

- Compare feature distributions: online (sampled) vs offline (training set)
- Statistical tests (KS test, PSI) on feature values
- Monitor model performance after each retraining
- Shadow scoring: run model on logged features vs live features

Prevention:

- Uber Michelangelo: bidirectional sync between online (Cassandra) and offline (Hive) stores
- Netflix DeLorean: same feature encoder code online and offline
- Root fix: **single source of truth** for feature computation

Detecting and Preventing Skew

Detection:

- Compare feature distributions: online (sampled) vs offline (training set)
- Statistical tests (KS test, PSI) on feature values
- Monitor model performance after each retraining
- Shadow scoring: run model on logged features vs live features

Prevention:

- Uber Michelangelo: bidirectional sync between online (Cassandra) and offline (Hive) stores
- Netflix DeLorean: same feature encoder code online and offline
- Root fix: **single source of truth** for feature computation

This leads us to a unified feature computation library.

Single Source of Truth for Features

Bottleneck: skew caused by separate online/offline code.

Solution: a **feature computation library** where each feature is *declared*:

- Computation code (the transformation logic)
- Input data sources and output schema
- Freshness guarantees (batch daily, streaming real-time)

Industry examples:

- **Snap Robusta:** YAML config with SQL transforms + aggregation specs
- **Airbnb Chronon:** Python declarations, generates lambda pipelines
- **Netflix DeLorean:** shared Java feature encoder classes
- **Uber Michelangelo:** Spark ML Transformer framework

Feature Declaration Examples

Robusta-style (YAML):

```
feature_group: user_engagement
  entity: user_id
  source: events.clicks
  aggregations:
    - name: click_count_7d
      function: COUNT
      window: 7d
```

Chronon-style (Python):

```
GroupBy(
    source=events.clicks,
    keys=["user_id"],
    aggregations=[
        Aggregation(
            input="*",
            operation=Operation.COUNT,
            windows=[Window(7, "d")]
        ),
    ],
)
```

Key: one declaration → generates both online and offline computation paths.

The Feature Store

The natural evolution: wrap the feature library in a *storage and serving platform*.

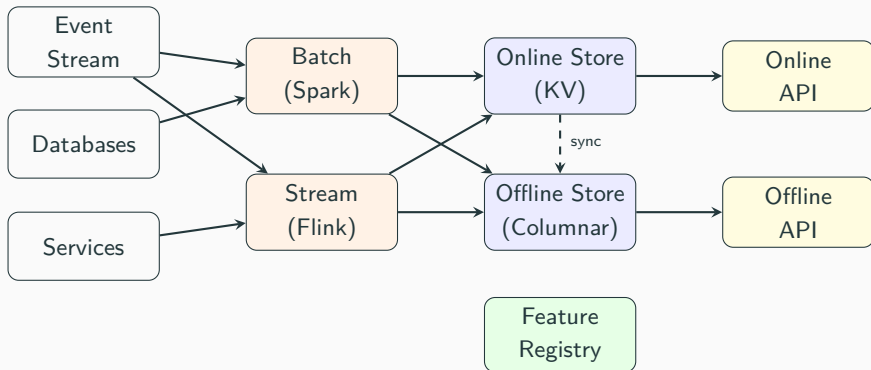
Two APIs:

1. **Online API:** return latest feature values for entity, low latency (<10 ms)
2. **Offline API:** return historical feature values in batch (point-in-time correct)

Core components:

- **Feature registry:** catalog of all features, their definitions, owners, lineage
- **Online store:** hot KV database for latest values
- **Offline store:** columnar storage for historical time-series
- **Materialization layer:** bridges online $\leftrightarrow$ offline
- **Computation engine:** executes feature transformations (batch + streaming)

Feature Store Architecture



Online Store: Choosing a Database

- **Redis** — in-memory, lowest latency, rich types; costly at scale (\$/GB)
- **Cassandra** — wide-column (LSM), multi-DC, per-cell TTL; JVM GC pauses
- **ScyllaDB** — Cassandra-compatible in C++, no GC; smaller ecosystem
- **DynamoDB** — fully managed (AWS), zero ops; unpredictable cost
- **Aerospike** — RAM index + SSD data, near-RAM latency; enterprise license

Architecture example: Venice (LinkedIn) — embedded RocksDB, microsecond reads via push-model. Shows how online store can be co-located with the service.

Pattern: online store receives $100\text{--}1000\times$ the QPS of the recommendation service due to per-candidate feature fan-out in ranking.

Offline Store: Table Formats on Object Storage

Foundation: Parquet/ORC files on S3/GCS (\$23/TB/month).

Table format provides: ACID transactions, time travel, schema evolution.

- **Apache Iceberg** — best metadata model, hidden partitioning; general purpose
- **Delta Lake** — Databricks-native, streaming + batch on same table

Critical operation: *point-in-time join* — retrieve features as they existed at a historical moment to avoid label leakage.

No major query engine natively supports ASOF JOIN — feature stores generate equivalent windowed SQL.

Materialization: Bridging Online and Offline

Problem: online and offline stores have different data models. Need a bridge.

Batch Materialization

- Spark/Airflow, hourly or daily
- Read latest values from offline store
- Write to online store (bulk upsert)
- Ensures completeness

Streaming Materialization

- Flink/Samza, seconds-level
- Process events in real-time
- Write to online store immediately
- Log back to offline for consistency

Lambda architecture: most production systems run *both* — batch ensures completeness, streaming ensures freshness.

Rule of thumb: streaming features cost 10–100× more to maintain than batch.

Feast: Open-Source Feature Store

Feast (Feature Store) — most widely adopted open-source feature store.

Core concepts:

- **Entity:** the join key (user_id, item_id, session_id)
- **Feature View:** a group of features from a single source, with schema
- **On-Demand Feature View:** transformations computed at request time
- **Data Source:** where raw data lives (BigQuery, Snowflake, Parquet, Kafka)

Architecture:

- Offline store: pluggable (BigQuery, Snowflake, Spark, DuckDB)
- Online store: pluggable (Redis, DynamoDB, SQLite)
- `feast materialize-incremental` bridges offline → online
- Python SDK for both online and offline retrieval

Netflix Approach: Snapshot Store

DeLorean (see earlier slide) — not a classic feature store, but a *snapshot store*.

Philosophy: don't store features — store the **world state** at each point in time.
Features are always computed on demand from raw snapshots.

How it fits the feature store picture:

- **Online store:** live microservice calls (no separate KV store)
- **Offline store:** Parquet snapshots on S3, indexed by time in Cassandra
- **Materialization:** not needed — same encoder code runs on live data or snapshots
- **Registry:** feature encoder classes *are* the registry

Trade-off: higher storage cost (raw snapshots > computed features), but maximum flexibility — any new feature can be computed from historical data without waiting.

Vector Databases in the Feature Store

Role: serve the **retrieval stage** — narrow billions of items to hundreds via ANN.

Libraries (embedded, no persistence):

- **Faiss** — IVF, PQ, HNSW, GPU. Maximum flexibility, build infra yourself.
- **ScaNN** — optimized for MIPS (dot-product), TensorFlow integration.

Databases (managed, with persistence):

- **Milvus** — broadest index support, microservices architecture, billion-scale
- **Qdrant** — best filtered search (during HNSW traversal), binary quantization
- **Pinecone** — zero-ops managed, seconds-fresh updates, vendor lock-in
- **Weaviate** — hybrid text + vector search
- **pgvector** — PostgreSQL extension, simplest to start, limited scale

Pros: Cross-Service Reusability

Before feature store: each service computes its own features in isolation.

After feature store: features are organizational assets.

Examples of cross-pollination:

- **Search** uses conversion signals originally computed for recommendations
- **Dynamic pricing** incorporates user engagement features from recs
- **Content team** uses engagement features for content acquisition decisions
- **Fraud detection** reuses user behavior patterns from personalization
- **Notifications** use predicted-interest features from the recommendation model

Multiplier effect: one well-maintained feature serves 5–10 downstream models.

Feature discovery via the registry prevents teams from reinventing the same signal.

Pros: Feature Store in the GenAI Era

New use cases beyond traditional features:

- **KV caches for transformers:** precomputed key-value attention states
→ store in online store, serve at inference time (PLUM, OneRec)
- **Embedding tables:** large-scale embedding lookups (DLRM-style)
→ feature store as the serving layer for embedding parameters
- **Prompt caches:** precomputed prompt embeddings for LLM-based rankers
- **User history sequences:** variable-length action histories for sequential models
→ stored as feature values, truncated/padded at retrieval time

Pattern: the feature store API generalizes to “return precomputed tensors for entity at time T ” — applicable far beyond scalar features.

Cons: Operational Cost

A feature store is a critical shared platform — it comes with obligations.

- **Dedicated engineering team:** 2–5 platform engineers to build and maintain
- **Development velocity:** adding features to the store has overhead vs ad-hoc
- **Single point of failure:** if the online store is down, all ranking models degrade
- **Monitoring & alerting:** feature freshness, latency, data quality, storage growth
- **On-call rotation:** someone must respond to feature store incidents 24/7
- **Migration cost:** moving existing features into the store is a multi-quarter project

When NOT to build:

- Small team (<5 ML engineers), few models
- Features don't need cross-team sharing
- Batch-only inference (no online serving)

Choosing the Data Platform: Managed

- **Databricks** — Managed FS, Delta Lake, advanced Spark (custom compilers, real-time), very costly
- **SageMaker FS** and **Vertex AI FS** — cloud-native FS for AWS / GCP respectively, good if already in ecosystem
- **Tracto** by Nebius — newer option for large scale and cheaper, significantly less adopted and very LLM-centric
- Fancy startups, trying to sell their new AI platform

Universal pattern: hot KV store (online) + columnar on object storage (offline) + sync layer.

Choosing the Data Platform: Custom

Custom stack (self-hosting, open-source, proprietary):

- **Redis / Cassandra + Iceberg on S3 (+ Feast)** — popular open-source data stack, custom FS solutions
- **YTsauros** — all-in-one OS data platform, from KV to queues to compute management

Build your own data stack:

- **3FS + Smallpond** — example of highly specialized tech, scale and domain dominate
- Do only if you **really** need it